

Neovim, built for reading code — with Claude in the loop.

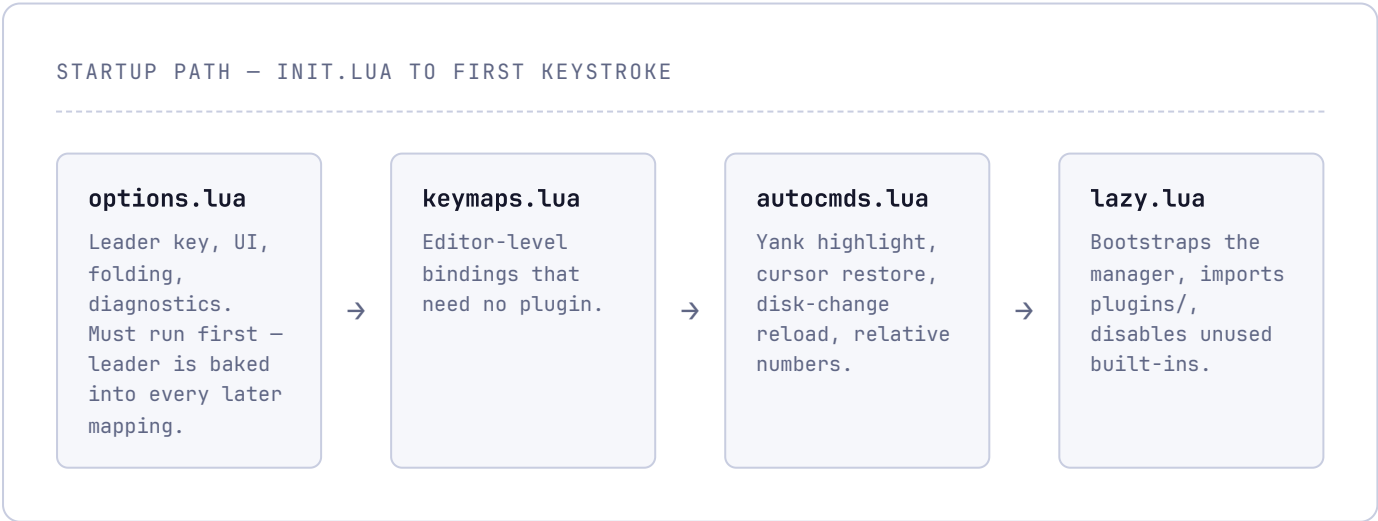
A 27-plugin Lua configuration targeting Neovim 0.11.2. The brief is organised around the question the config was built to answer: how fast can you understand code you did not write?

27 PLUGINS	5 LOADED AT STARTUP	22 DEFERRED	9 LSP SERVERS	37 TS PARSERS
2 AI INTEGRATIONS				

01 — ARCHITECTURE

How the config is assembled

Four plain Lua modules run in a fixed order, then `lazy.nvim` takes over and imports one file per concern from `lua/plugins/`. Nothing is clever: each file owns one subject, and the plugin manager resolves the rest.



One file per concern

~/.config/nvim

```
init.lua          4 requires, nothing else
lua/config/
  options · keymaps · autocmds · lazy
lua/plugins/
  ui.lua          theme, statusline, tabs, dashboard, which-key
  explore.lua     file tree, outline, motion, pairs
  treesitter.lua  syntax, textobjects, sticky context
  fzf-lua.lua     every picker
  lsp.lua         mason + lspconfig + blink.cmp
  lint.lua        nvim-lint
  git.lua         gitsigns
  terminal.lua    FTerm float + lazygit
  ai.lua          claudecode + codecompanion
```

Why 27 plugins still start instantly

Only five plugins load when Neovim opens. The other twenty-two sit dormant until something asks for them — a keypress, a command, a filetype, or the `VeryLazy` event that fires after the UI is already drawn.

STARTUP COST — EAGER VS DEFERRED

5 EAGER**22 DEFERRED**

■ colorscheme · treesitter · nvim-tree · lazy · devicons ■ loaded on demand

The four triggers

WHAT WAKES A PLUGIN

keys

A mapping is registered as a stub. Press it once and the real plugin loads, then the key fires. This is how `fzf-lua`, `aerial`, `flash` and both AI tools stay out of startup.

cmd

A command name is reserved. Typing `:Mason` or `:Lazygit` pulls the plugin in on first use.

event

`BufReadPre` for LSP and `gitsigns`, `InsertEnter` for completion and autopairs — nothing loads before a real buffer exists.

VeryLazy

Fires after the first screen is painted. Statusline, bufferline and which-key appear a few milliseconds late and nobody notices.

Three layers that understand your code

These are frequently confused. They solve different problems and run independently — treesitter works with no language server attached, and the language server works with no parser installed.

SEPARATION OF CONCERNS

Treesitter

Structure of this file. Parses the buffer into a syntax tree on every keystroke. Gives you accurate highlighting, folding, the sticky function header, and the `vaf / ]f` motions that operate on functions rather than lines. Instant, local, no network.

LSP

Meaning across the project. A background server per language that resolves symbols. Everything that needs to know about *other* files — go-to-definition, references, rename, diagnostics, call hierarchy — comes from here.

blink.cmp

Presentation. Merges completion candidates from the language server, snippets, open buffers and file paths into one ranked menu. Owns no knowledge of its own.

nvim-lint

Opinions the server does not hold. Runs standalone linters — shellcheck, hadolint, golangci-lint — on write. Silently skips any whose binary is absent.

WHERE SERVERS COME FROM

mason downloads the binaries; `mason-lspconfig` enables them. The enable list is an explicit allow-list in `lua/plugins/lsp.lua` — not `true` — because mason also holds formatters, and a formatter started as a language server crashes on every launch.

Every plugin, and why it's there

READING & NAVIGATION The reason the config exists

fzf-lua

Every picker: files, live grep, buffers, git, and all the LSP result lists. Wraps the fzf binary, so it stays fast on large repositories.

```
<leader>ff · <leader>/
```

nvim-treesitter

Syntax tree per buffer. Drives highlighting, indentation and folding.

```
<C-space> to grow selection
```

nvim-treesitter-textobjects

Turns functions, classes and arguments into things you can select and jump between.

```
vaf · ]f · [c
```

nvim-treesitter-context

Pins the enclosing function signature to the top of the window while you scroll. Quietly the highest-value plugin here.

```
[x to jump up
```

aerial.nvim

Symbol outline sidebar. The fastest way to get oriented in a long unfamiliar file.

```
<leader>o
```

nvim-tree.lua

Project file tree with git status and diagnostics shown per file.

```
<leader>e
```

flash.nvim

Type two characters plus a label to jump anywhere on screen. Replaces most searching.

```
s
```

CODE INTELLIGENCE Definitions, completion, diagnostics

nvim-lspconfig

Server definitions consumed by Neovim 0.11's native `vim.lsp.config` / `vim.lsp.enable`.

```
gd · gr · <leader>cr
```

mason.nvim

Installs language servers, linters and formatters without touching system package managers.

```
<leader>cm
```

mason-lspconfig.nvim

Bridges the two — installs the servers listed in SERVERS and enables exactly those.

—

blink.cmp

Completion menu with a Rust fuzzy matcher. Pinned to v1 while v2 stabilises.

<C-y> accepts

friendly-snippets

Snippet corpus feeding blink's snippet source.

<Tab> between fields

nvim-lint

Async linters on write, filtered to those actually installed.

<leader>cℓ

AI Two tools, deliberately separate

claudecode.nvim

Runs the Claude CLI in a split using the same protocol as the VS Code extension. Sends selections as context; returns edits as reviewable Neovim diffs. Uses your subscription — no API key.

<leader>ac

codecompanion.nvim

Native chat buffer and inline assistant against the Anthropic API. Requires ANTHROPIC_API_KEY; billed per token.

<leader>nn

GIT & TERMINAL Change review and shell access

gitsigns.nvim

Gutter signs, hunk staging and reset, inline blame, and a hunk text object.

]h · <leader>gb

FTerm.nvim

Floating terminal. Also hosts lazygit, which removes the need for a separate git-UI plugin.

<C-\\> · <leader>gg

INTERFACE Theme, chrome, discoverability

catppuccin

Mocha flavour, transparent background so the terminal shows through.

—

lualine.nvim

Statusline: mode, branch, diagnostics, relative path, attached LSP servers.

—

barbar.nvim

Buffer tabs with pinning, reordering and jump-by-letter.

`<S-h> · <S-l>`

which-key.nvim

Press leader and pause — every binding appears, grouped. The config's own documentation.

`<leader>`

alpha-nvim

Start screen with direct routes into files, grep, tree, lazygit and Claude.

—

nvim-colorizer.lua

Renders hex and rgb values as swatches inline.

`<leader>uc`

nvim-web-devicons

Filetype glyphs. Requires a Nerd Font in the terminal.

—

nvim-autopairs

Closes brackets and quotes, treesitter-aware.

—

lazy.nvim

The plugin manager. Owns the lockfile and the deferred-loading model.

`<leader>L`

plenary.nvim

Shared Lua utility library. A dependency, never used directly.

—

The keys that matter

Leader is `Space`. You never need this table in practice — press leader and wait, and which-key shows the same information in context. These are the ones worth committing to muscle memory.

Finding things

<code>Spc</code> <code>Spc</code>	Find files
<code>Spc</code> <code>/</code>	Live grep across the project
<code>Spc</code> <code>s</code> <code>w</code>	Grep the word under the cursor
<code>Spc</code> <code>s</code> <code>r</code>	Reopen the last picker, with its query
<code>Spc</code> <code>e</code>	File tree
<code>Spc</code> <code>o</code>	Symbol outline
<code>s</code>	Flash jump to anything visible

Understanding code

<code>g</code> <code>d</code>	Go to definition
<code>g</code> <code>r</code>	Find all references
<code>g</code> <code>I</code> · <code>g</code> <code>y</code>	Implementation · type definition
<code>K</code>	Hover documentation
<code>Spc</code> <code>c</code> <code>c</code>	Incoming calls — who calls this?
<code>C-Space</code>	Grow selection by syntax node; repeat to widen
<code>v</code> <code>a</code> <code>f</code>	Select the whole enclosing function
<code>]</code> <code>f</code> · <code>[</code> <code>f</code>	Jump to next / previous function

Changing code

<code>Spc</code> <code>c</code> <code>r</code>	Rename symbol everywhere
<code>Spc</code> <code>c</code> <code>a</code>	Code action
<code>Spc</code> <code>c</code> <code>f</code>	Format buffer
<code>]</code> <code>d</code> · <code>[</code> <code>d</code>	Next / previous diagnostic

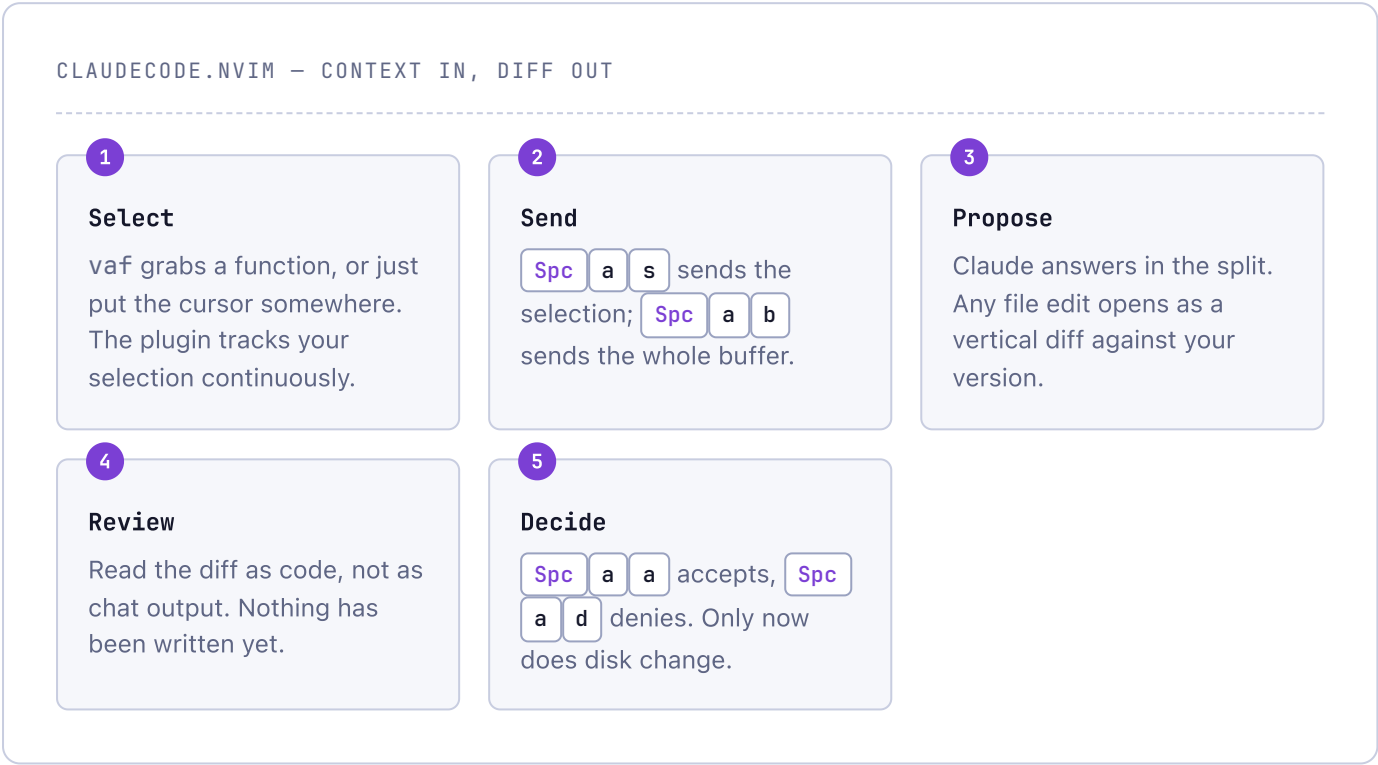
Spc	s	R	Start a substitution
Spc	?		Show this buffer's bindings

Git

Spc	g	g	Lazygit, full screen
] h	.	[h	Next / previous changed hunk
Spc	g	h	Stage the hunk under the cursor
Spc	g	b	Blame this line, with the full message
Spc	g	c	This file's commit history
Spc	g	d	Diff against the index

The review loop

The design principle is that Claude never silently rewrites a buffer. Every edit it proposes surfaces as a diff you explicitly accept or reject, in the same window where you were already reading the code.



The full binding set

Spc a c	Toggle the Claude split
Spc a s	Send selection — or, in the tree, add that file
Spc a b	Add the current buffer as context
Spc a r	Resume an earlier session
Spc a a	Accept the proposed diff
Spc a d	Deny the proposed diff
Spc a m	Switch model
Spc n n	CodeCompanion chat (needs API key)

WHY BUFFERS RELOAD ON THEIR OWN

Claude and lazygit both write to disk, not to your open buffer. An autocommand runs `checktime` on focus and cursor-hold so the buffer refreshes and announces it. Without that, your next `:w` would overwrite their work with stale text.

What will break, and when

Three plugins are pinned to older branches because their defaults now require Neovim 0.12. These are not arbitrary version locks — unpinning them on 0.11.2 breaks the editor on launch.

<code>nvim-treesitter</code>	<code>branch = "master"</code> — <code>main</code> is a full rewrite with an incompatible API. Migrate, don't unpin.
<code>nvim-treesitter-textobjects</code>	<code>branch = "master"</code> — same rewrite.
<code>aerial.nvim</code>	<code>branch = "nvim-0.11"</code> — the maintained compatibility branch.
<code>blink.cmp</code>	<code>version = "1.*"</code> — v2 is still landing breaking changes.

KNOWN ROUGH EDGES

No formatter. `<leader>cf` uses LSP formatting, which is good for Go and Lua and weak for JS, TS and Python. `conform.nvim` would close this gap.

CodeCompanion is inert until `ANTHROPIC_API_KEY` is exported. Claude Code needs no key and covers most of the same ground.

`s` is taken by flash. Use `c` `l` for the old substitute-character behaviour.

ICONS DEPEND ON THE TERMINAL, NOT THE CONFIG

Every glyph in the statusline, tree and completion menu requires a Nerd Font set in the terminal emulator itself. Warp reads `[appearance.text] font_name` in `~/.warp/settings.toml`; Alacritty reads `alacritty.toml`. Setting one does nothing for the other.